

МІНІСТЕРСТВО ОСВІТИ І НАУКИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ імені ІВАНА
ФРАНКА
Факультет прикладної математики та інформатики
Кафедра програмування

Лабораторна робота №10
Додаткові графічні операції

Виконала:
Студентка групи ПМІ-46
Тригуб Марта

Мета роботи

Завершити розробку графічного редактора реалізацією трьох груп додаткових операцій: смуг прокрутки для роботи з великими малюнками, що не вміщаються у вікні; піксельних фільтрів зображення (інверсія, відтінки сірого, розмиття, контур); геометричних перетворень усього малюнка (обертання на кути кратні 90°, дзеркальне відображення).

Теоретичні відомості

Смуги прокрутки у tkinter Canvas

Компонент Canvas у tkinter підтримує концепцію «scrollregion» – логічну область полотна, яка може бути більшою за видиму частину вікна. Смуги прокрутки (Scrollbar) зв'язуються з canvas через параметри xscrollcommand/yscrollcommand та методи xview/yview. Важливою особливістю є те, що події миші повертають координати у системі відліку вікна, а не полотна — тому для коректного малювання потрібно перетворювати координати через `canvas.canvasx()` і `canvas.canvasy()`, які враховують поточне зміщення смуг прокрутки.

Фільтри зображення та бібліотека Pillow

Піксельні фільтри – це операції, що обробляють кожен піксел зображення на основі його значення та значень сусідніх пікселів. Для реалізації фільтрів у Python використовується бібліотека Pillow (PIL). Вона надає модуль ImageFilter з готовими ядрами згортки (Gaussian Blur, FIND_EDGES) та ImageOps для простих поканальних перетворень (Invert, Grayscale). Оскільки редактор використовує векторну графіку, перед застосуванням фільтру canvas растеризується через PIL ImageDraw, а результат відображається поверх canvas як PhotoImage.

Геометричні перетворення координат

Обертання точки (x, y) навколо центру (cx, cy) на кут α виконується за формулами: $x' = cx + (x - cx) \cdot \cos \alpha - (y - cy) \cdot \sin \alpha$, $y' = cy + (x - cx) \cdot \sin \alpha + (y - cy) \cdot \cos \alpha$. Оскільки полотно є векторним, достатньо застосувати формулу до координат x_1, y_1, x_2, y_2 кожної фігури і перемалювати. Дзеркальне відображення горизонтально: $x' = 2 \cdot cx - x$ (y незмінне); вертикально: $y' = 2 \cdot cy - y$.

Опис програмної реалізації

Завдання 1 – Смуги прокрутки

Полотно розміром 2000×2000 пікселів із горизонтальною і вертикальною смугами прокрутки реалізовано у методі `_build_canvas()`:

```
def _build_canvas(self):  
    frame = tk.Frame(self.root, (parameter) expand: bool | Literal[0, 1]  
    frame.pack(fill=tk.BOTH, expand=True)  
  
    # вертикальна і горизонтальна смуги прокрутки  
    vbar = tk.Scrollbar(frame, orient=tk.VERTICAL)  
    vbar.pack(side=tk.RIGHT, fill=tk.Y)  
  
    hbar = tk.Scrollbar(frame, orient=tk.HORIZONTAL)  
    hbar.pack(side=tk.BOTTOM, fill=tk.X)  
  
    # велике полотно  
    self.canvas = tk.Canvas(  
        frame,  
        bg="white",  
        cursor="crosshair",  
        scrollregion=(0, 0, CANVAS_W, CANVAS_H),  
        xscrollcommand=hbar.set,  
        yscrollcommand=vbar.set,  
    )
```

Для прокрутки колесом миші додано обробники `MouseWheel` (Windows) та `Button-4/Button-5` (Linux). `Shift`+колесо прокручує горизонтально. У всіх обробниках подій миші координати тепер отримуються через метод `_canvas_coords()`, який враховує поточне зміщення смуг:

```
def _canvas_coords(self, event):  
    x = self.canvas.canvasx(event.x)  
    y = self.canvas.canvasy(event.y)  
    return x, y
```

Метод `_canvas_coords()` викликається у всіх обробниках `on_mouse_down`, `on_mouse_move`, `on_mouse_up`, `on_mouse_hover` – це забезпечує коректну роботу всіх операцій малювання і виокремлення на будь-якій ділянці великого полотна після прокрутки.

Завдання 2 – Фільтри зображення

Реалізовано чотири фільтри: Інверсія, Відтінки сірого, Розмиття, Контур. Основний метод `apply_filter()` спочатку растеризує `canvas` через `_rasterize_canvas()`, потім застосовує фільтр і відображає результат:

```
def apply_filter(self, filter_name: str):
    if not PIL_AVAILABLE:
        messagebox.showerror("Помилка", "Бібліотека Pillow не встановлена.")
        return
    img = self._rasterize_canvas()
    if img is None:
        return

    # застосування фільтру
    if filter_name == "invert":
        result = ImageOps.invert(img)
    elif filter_name == "grayscale":
        result = ImageOps.grayscale(img).convert("RGB")
    elif filter_name == "blur":
        result = img.filter(ImageFilter.GaussianBlur(radius=3))
    elif filter_name == "edges":
        # Контур: FIND_EDGES на градаціях сірого, потім повернути у RGB
        gray = ImageOps.grayscale(img)
        edges = gray.filter(ImageFilter.FIND_EDGES)
        result = edges.convert("RGB")
    else:
        return
```

Растеризація виконується через PIL `ImageDraw`: для кожної фігури у `self.shapes` викликаються `draw.line()`, `draw.rectangle()` або `draw.ellipse()` з відповідними параметрами кольору і товщини. Фільтр-шар розміщується нижче векторних фігур (`tag_lower`), тому фігури залишаються видимими. Метод `remove_filter()` видаляє `photo-image` з `canvas` і звільняє пам'ять.

Фільтр	Метод Pillow	Ефект
Інверсія	<code>ImageOps.invert()</code>	Інвертує кожен канал RGB: $R' = 255 - R$
Відтінки сірого	<code>ImageOps.grayscale()</code>	Перетворює у яскравість: $Y = 0.299R + 0.587G + 0.114B$

Розмиття	GaussianBlur(radius=3)	Гаусове розмиття із радіусом 3px
Контур	FIND_EDGES	Згортка ядром Собеля для виділення меж

Завдання 3 – Обертання і дзеркало

Метод `rotate_all(angle_deg)` застосовує матрицю обертання до координат усіх фігур навколо центру полотна (1000, 1000):

```
def rotate_all(self, angle_deg: int):
    """Обернути всі фігури навколо центру їх bounding box."""
    if not self.shapes:
        return
    self._push_undo()
    bx1, by1, bx2, by2 = self._shapes_bbox()
    cx, cy = (bx1 + bx2) / 2, (by1 + by2) / 2

    # Для кратних 90° округлюємо тригонометрію до цілих, щоб уникнути float-сміття
    rad = math.radians(angle_deg)
    cos_a = round(math.cos(rad))
    sin_a = round(math.sin(rad))

    def rot(x, y):
        dx, dy = x - cx, y - cy
        return round(cx + dx * cos_a - dy * sin_a, 2), round(cy + dx * sin_a + dy * cos_a, 2)

    for s in self.shapes:
        s["x1"], s["y1"] = rot(s["x1"], s["y1"])
        s["x2"], s["y2"] = rot(s["x2"], s["y2"])

    self._redo_stack.clear()
    self._redraw_all()
    self.is_modified = True
    self.status_var.set(f"Обернуто на {angle_deg}°")
```

Метод `mirror_all(axis)` виконує дзеркальне відображення: для горизонтального — $x' = 2 \cdot cx - x$, для вертикального — $y' = 2 \cdot cy - y$. Обидві операції зберігаються у стеку Undo.

Нові елементи меню і тулбару

Елемент	Дія
Меню Фільтри > Інверсія	Інвертувати кольори растрованого полотна
Меню Фільтри > Відтінки сірого	Перетворити у градації сірого

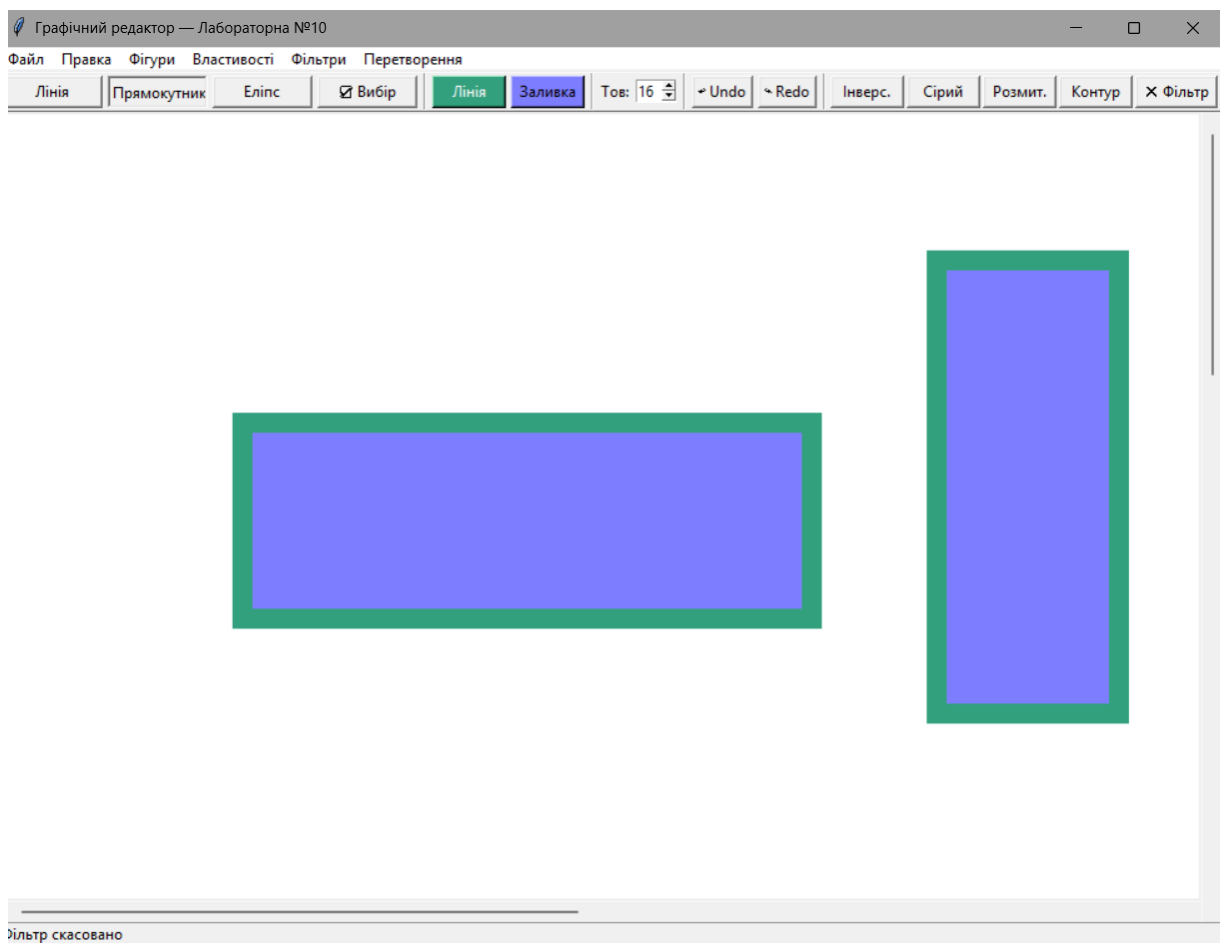
Меню Фільтри > Розмиття	Застосувати Гаусове розмиття (r=3)
Меню Фільтри > Контур	Виділення контурів через FIND_EDGES
Меню Фільтри > Скасувати фільтр	Прибрати шар фільтру
Меню Перетворення > Повернути 90° за год.	Обертання усіх фігур на +90°
Меню Перетворення > Повернути 90° проти год.	Обертання на -90°
Меню Перетворення > Повернути на 180°	Обертання на 180°
Меню Перетворення > Дзеркало горизонтально	Відображення по осі X
Меню Перетворення > Дзеркало вертикально	Відображення по осі Y
Кнопки тулбару    	Відповідні швидкі кнопки для перетворень

Тестування

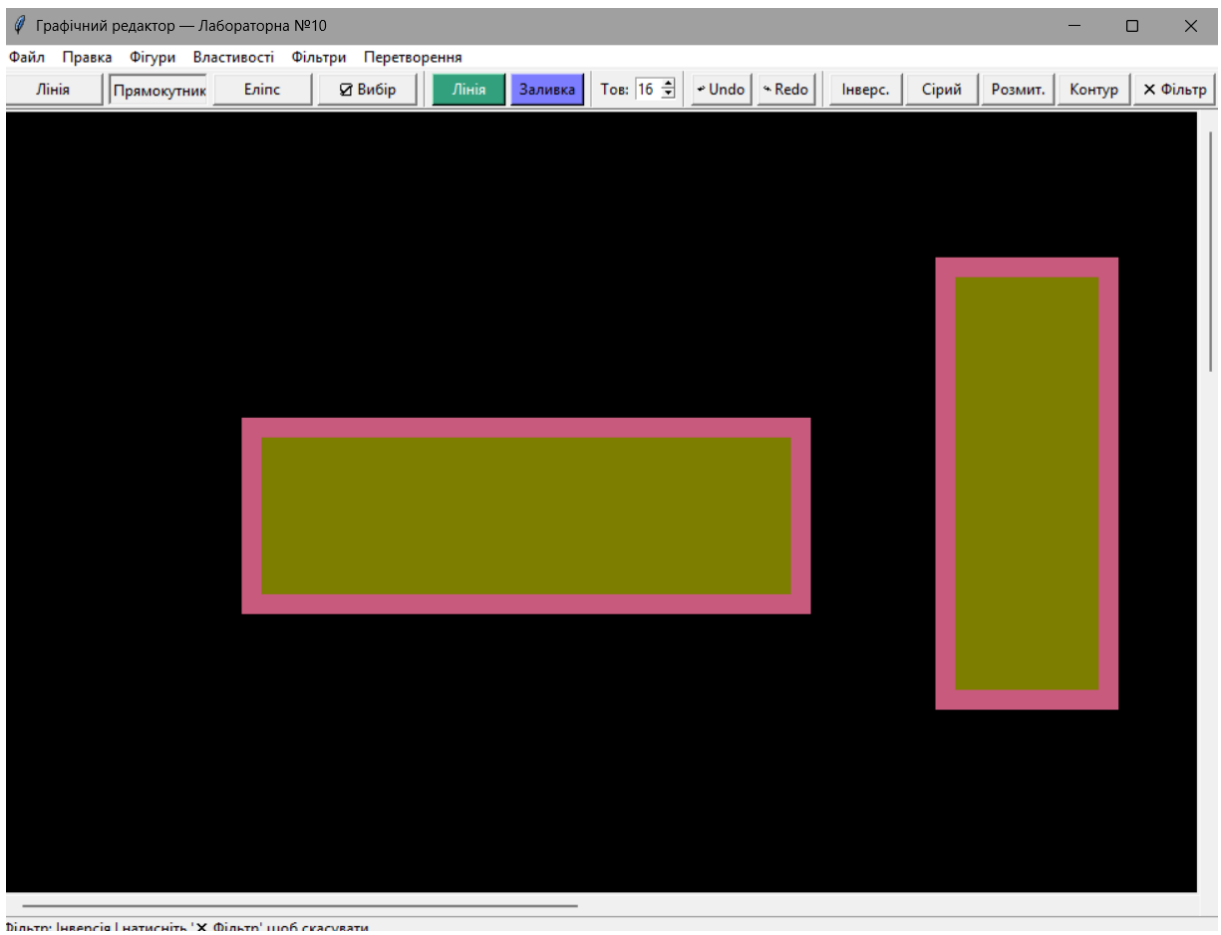
1. Намалювати фігури у центрі видимої частини полотна, прокрутити вниз і праворуч – фігури відображаються коректно. Намалювати нові фігури після прокрутки – координати визначаються правильно завдяки `_canvas_coords()`.
2. Намалювати кілька кольорових фігур -> Фільтри > Інверсія – кольори інвертовані, вектори залишаються поверх. Натиснути 'X Фільтр' – оригінальний вигляд відновлено.
3. Фільтр 'Відтінки сірого' коректно перетворює кольорові фігури у сіру гаму. Фільтр 'Розмиття' зглажує краї. Фільтр 'Контур' виводить тільки межі фігур на чорному фоні.
4. Намалювати прямокутник -> Перетворення > Повернути 90° – прямокутник повертається навколо центру полотна. Ctrl+Z відновлює початкове положення.

5. Дзеркало горизонтально: фігури у лівій половині переходять у праву і навпаки. Дзеркало вертикально: верх і низ міняються місцями.
6. Зберегти малюнок після обертання -> відкрити -> всі фігури відтворені з новими координатами.

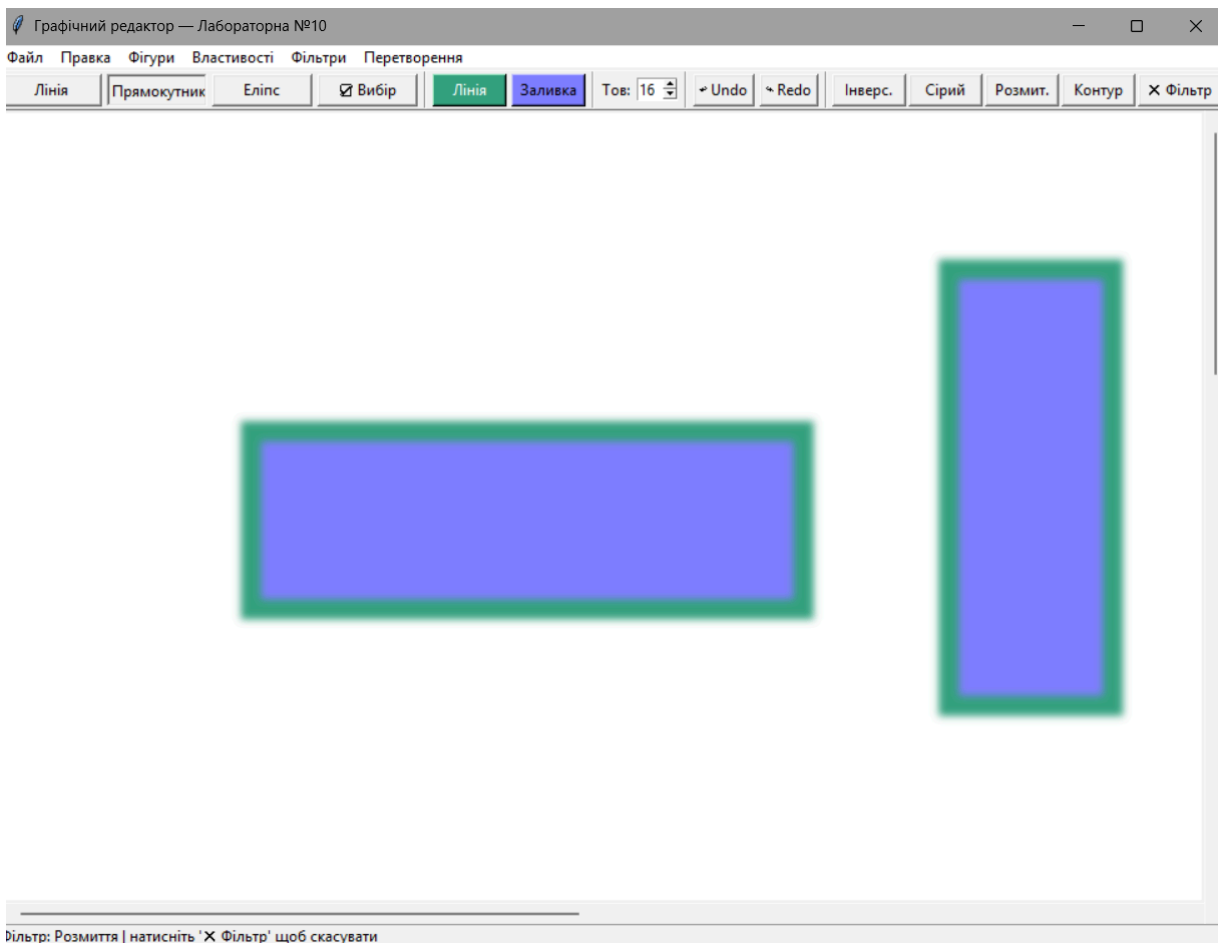
Всі сценарії виконано успішно. Прокрутка і малювання коректно взаємодіють. Фільтри відображаються миттєво і не впливають на векторні дані. Обертання і дзеркало точно трансформують координати фігур.



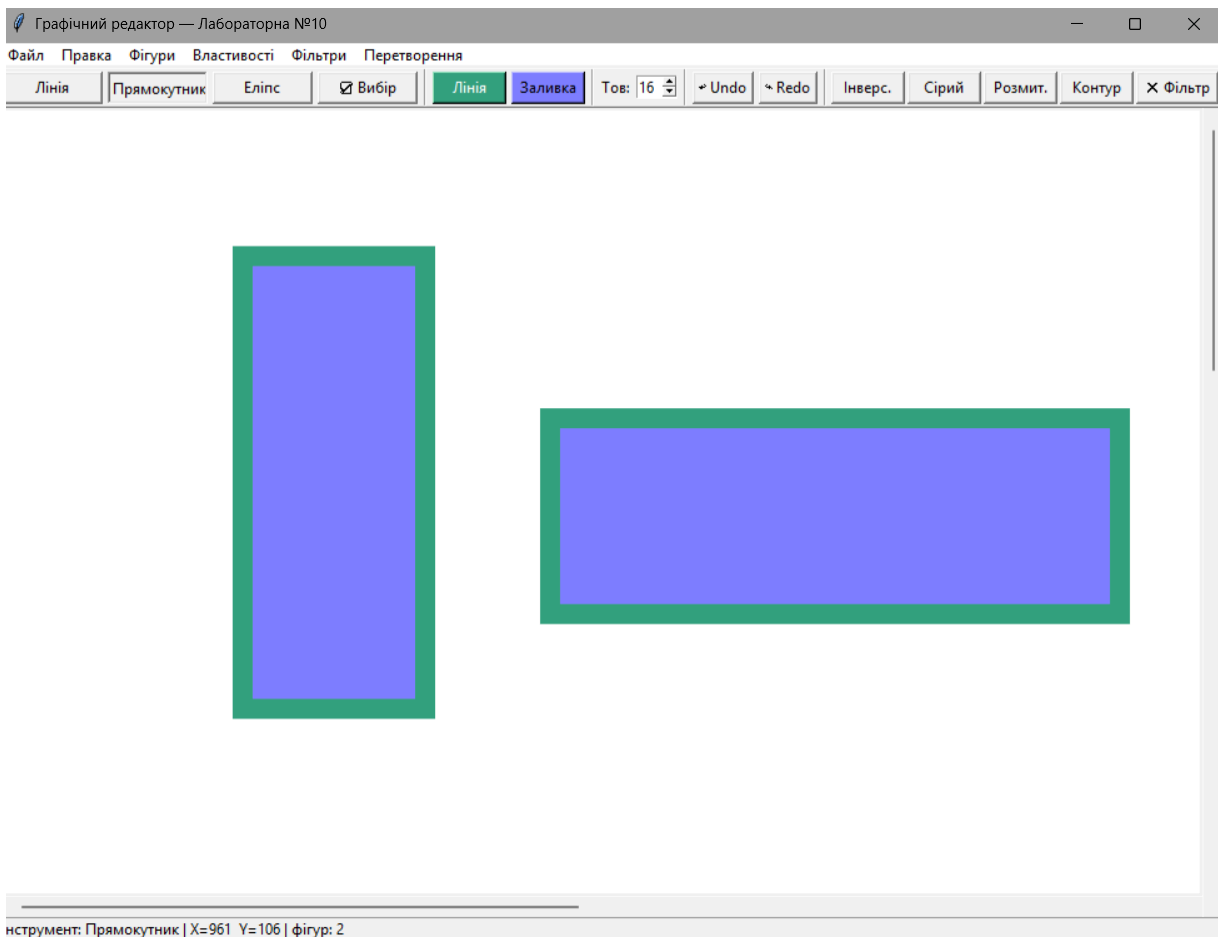
Оригінальний вигляд фігур



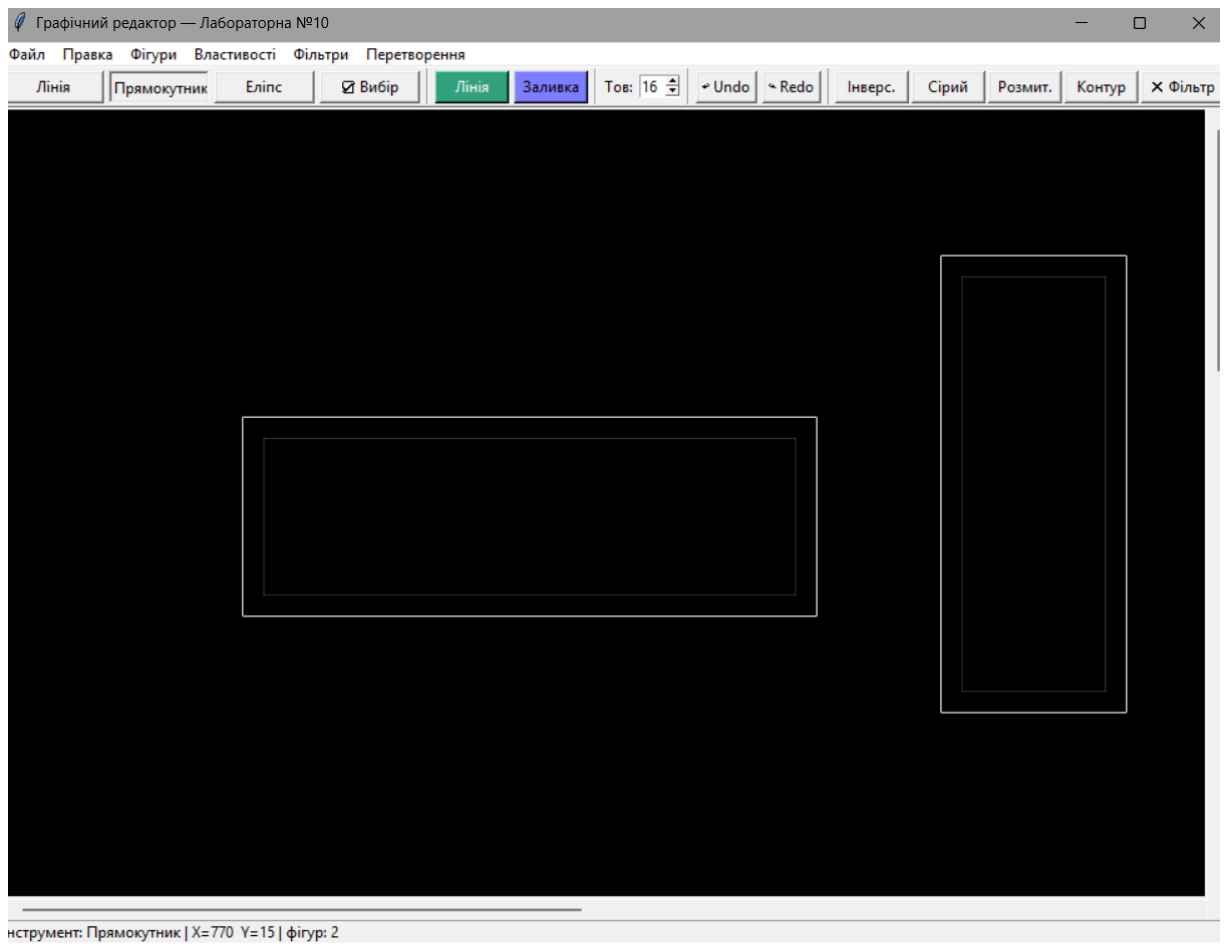
Інверсія



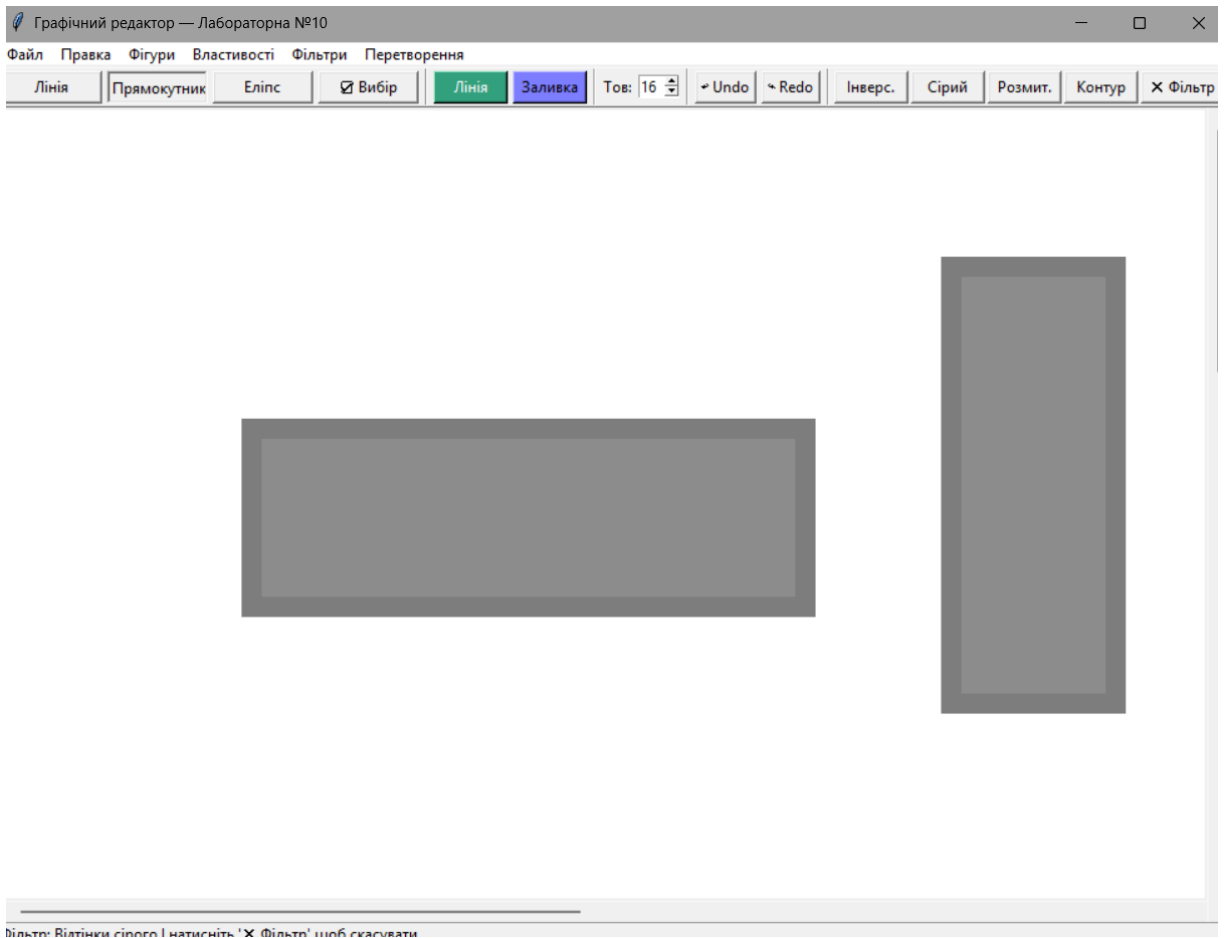
Розмиття



Дзеркало горизонтально



Контур



Відтінки сірого

Висновки

У ході трьох лабораторних робіт (8, 9, 10) розроблено повнофункціональний векторний графічний редактор мовою Python із використанням tkinter і Pillow. У лабораторній роботі №10 реалізовано підтримку великих малюнків через смуги прокрутки з коректним перерахунком координат миші; піксельні фільтри зображення на базі бібліотеки Pillow (інверсія, сірий, розмиття, контур); а також геометричні перетворення усього малюнка – обертання та дзеркальне відображення – реалізовані аналітично через матрицю обертання.

Поєднання векторного підходу із растеризацією для фільтрів дозволяє отримати переваги обох методів: векторні дані зручно редагувати і трансформувати, а PIL надає потужні пікселеві операції. Завершений редактор демонструє повний цикл проектування графічного застосунку методом низхідного програмування (top-down):

від порожнього каркасу до готового інструменту з розширеними можливостями.